

Main chapter ko concept-by-concept break kar raha hoon: closure ki exact condition, live-variable behavior, loop trap, callbacks, GC/memory, aur practical design benefits. Examples ko simpler execution steps ke saath explain karunga. # Chapter 7: Using Closures, Detailed Hinglish Explanation

Is chapter ka main purpose closure ko sirf definition ki tarah yaad karna nahi, balki **program me closure ko identify karna, correctly use karna, aur uske memory impact ko samajhna** hai. Closure lexical scope ke upar build hota hai, isliye agar scope clear hai, closure bhi gradually clear ho jayega.

1. Closure ki need kyon padti hai?

Pichhle chapters ka important principle tha:

Variables ko jitne chhote scope me rakh sakte ho, utne chhote scope me rakho.

Isse:

- accidental name collision kam hota hai
- global variables kam hote hain
- code samajhna easier hota hai
- variables galat jagah se modify nahi hote
- code maintainable rehta hai

Lekin problem ye hai ki kabhi-kabhi hume kisi local variable ki value ko **future me bhi use karna hota hai**, outer function ke complete ho jaane ke baad bhi.

Closure isi problem ko solve karta hai.

Closure ki help se ek function apne outer scope ke variables ko future ke liye preserve karke access kar sakta hai.

Simple mental model:

```
1 Outer function complete ho gaya
2     ↓
3 Uska inner function abhi bhi available hai
4     ↓
5 Inner function ko outer variables yaad hain
6     ↓
7 Ye closure hai
```

2. Closure sirf functions ke saath hota hai

Chapter ek important distinction deta hai:

- Object khud closure nahi rakhta
- Class khud closure nahi rakhti
- Function aur class ke methods closure rakh sakte hain
- Closure function ka behavior hai

Closure hone ke liye generally teen conditions chahiye:

1. Ek function hona chahiye
 2. Function kisi outer scope ke variable ko reference kare
 3. Function ko us outer scope ke complete hone ke baad ya kisi doosri scope-chain branch se invoke kiya jaye
-

3. First closure example

```
1  function lookupStudent(studentID) {
2      var students = [
3          { id: 14, name: "Kyle" },
4          { id: 73, name: "Suzy" },
5          { id: 112, name: "Frank" },
6          { id: 6, name: "Sarah" }
7      ];
8
9      return function greetStudent(greeting) {
10         var student = students.find(
11             student => student.id == studentID
12         );
13
14         return `${greeting}, ${student.name}!`;
15     };
16 }
17
18 var chosenStudents = [
19     lookupStudent(6),
20     lookupStudent(112)
21 ];
22
```

```
23 chosenStudents"Hello"; // Hello, Sarah!  
24 chosenStudents"Howdy"; // Howdy, Frank!
```

Step-by-step execution

First call

```
1 lookupStudent(6)
```

Is call ke liye ek naya scope create hota hai:

```
1 studentID = 6  
2 students = [...]
```

Phir lookupStudent() ye inner function return karta hai:

```
1 function greetStudent(greeting) {  
2     // ...  
3 }
```

Returned function ko chosenStudents[0] me save kar diya jata hai.

Important baat:

```
1 lookupStudent(6)
```

complete ho chuka hai, fir bhi returned greetStudent() function ko:

- studentID
- students

donon variables accessible hain.

Second call

```
1 lookupStudent(112)
```

Is call ke liye ek completely separate scope create hota hai:

```
1 studentID = 112  
2 students = [...]
```

Aur ek naya greetStudent() function instance create hota hai.

Isliye:

```
1 chosenStudents[0]
```

apne scope ke studentID = 6 ko remember karta hai.

Aur:

```
1 chosenStudents[1]
```

apne alag scope ke studentID = 112 ko remember karta hai.

Agar closure nahi hota

lookupStudent() complete hote hi ideally uske local variables cleanup ho jaate:

```
1 studentID
2 students
```

Phir later greetStudent() unhe access karta to ReferenceError aana chahiye tha.

Lekin error nahi aata:

```
1 chosenStudents"Hello";
2 // Hello, Sarah!
```

Ye observable proof hai ki closure ne required variables ko preserve kiya.

4. Closure exactly kis function ke paas hai?

Previous example me ye callback dekho:

```
1 students.find(
2     student => student.id == studentID
3 );
```

Arrow function bhi function hai:

```
1 student => student.id == studentID
```

Isliye arrow function ka apna function scope hai.

studentID ko directly arrow function access kar raha hai. Is wajah se technically studentID par closure arrow function hold karta hai.

Scopes approximately aise hain:

```
1 Global scope
2   └─ lookupStudent scope
3       └─ studentID
4       └─ students
5           └─ greetStudent scope
6               └─ greeting
7               └─ student
8                   └─ arrow function scope
9                       └─ student parameter
10                          └─ outer studentID ka access
```

Ek aur important point:

```
1 var student = students.find(
```

```
2     student => student.id == studentID
3 );
```

Yahan do alag student hain:

1. Outer var student
2. Arrow-function parameter student

Arrow function ka parameter outer student ko shadow karta hai.

Ye legal hai, kyunki dono different scopes me hain.

5. Har function instance ka apna closure hota hai

```
1  function adder(num1) {
2      return function addTo(num2) {
3          return num1 + num2;
4      };
5  }
6
7  var add10To = adder(10);
8  var add42To = adder(42);
9
10 add10To(15); // 25
11 add42To(9);  // 51
```

First execution

```
1  adder(10)
```

Is execution me:

```
1  num1 = 10
```

Ek naya addTo() function instance create hota hai. Ye apne execution wale num1 ko close over karta hai.

```
1  var add10To = adder(10);
```

Conceptually:

```
1  add10To = function addTo(num2) {
2      return 10 + num2;
3  };
```

Lekin technically 10 direct function me copy nahi hua. Function num1 variable ko access kar raha hai.

Second execution

```
1 adder(42)
```

Ek naya scope aur naya num1 create hota hai:

```
1 num1 = 42
```

Aur ek alag addTo() function instance banta hai.

Isliye:

```
1 add10To(15); // 10 + 15
2 add42To(9);  // 42 + 9
```

Important conclusion

Closure sirf function ki lexical definition ke saath associated nahi hota.

Closure **har function instance** ke saath associated hota hai.

```
1 adder(10)
2   → addTo instance 1
3   → num1 instance 1 = 10
4
5 adder(42)
6   → addTo instance 2
7   → num1 instance 2 = 42
```

Function definition source code me ek baar hai:

```
1 function addTo(num2) {
2     return num1 + num2;
3 }
```

Lekin runtime par uske multiple instances ban sakte hain, aur har instance ka separate closure ho sakta hai.

6. Closure snapshot nahi, live link hai

Ye closure ka sabse important concept hai.

Closure kisi variable ki current value ko photograph ki tarah capture nahi karta. Closure actual variable ke saath **live connection** preserve karta hai.

Matlab function:

- variable ko read kar sakta hai
- variable ko update kar sakta hai

- future me uski latest value dekh sakta hai

Example:

```
1 function makeCounter() {  
2     var count = 0;  
3  
4     return function getCurrent() {  
5         count = count + 1;  
6         return count;  
7     };  
8 }  
9  
10 var hits = makeCounter();  
11  
12 hits(); // 1  
13 hits(); // 2  
14 hits(); // 3
```

Execution

```
1 var hits = makeCounter();
```

makeCounter() ke scope me:

```
1 count = 0
```

Inner getCurrent() function return ho jata hai aur hits me save hota hai.

Ab:

```
1 hits();
```

inner function preserved count ko access karta hai:

```
1 count = count + 1;
```

First call:

```
1 0 → 1
```

Second call:

```
1 1 → 2
```

Third call:

```
1 2 → 3
```

Agar closure snapshot hota, har call ko count = 0 milta aur result hamesha 1 hota.

Lekin actual result:

```
1 1
```

```
2 2
3 3
```

Ye prove karta hai ki closure variable ka previous updated state preserve karta hai.

7. Outer scope function hona required nahi

Usually closure me ek outer function hota hai:

```
1 function outer() {
2     let value = 10;
3
4     return function inner() {
5         return value;
6     };
7 }
```

Lekin outer scope ka function scope hona compulsory nahi hai. Block scope bhi ho sakta hai:

```
1 var hits;
2
3 {
4     let count = 0;
5
6     hits = function getCurrent() {
7         count = count + 1;
8         return count;
9     };
10 }
11
12 hits(); // 1
13 hits(); // 2
14 hits(); // 3
```

Block complete hone ke baad normally count inaccessible ho jana chahiye:

```
1 {
2     let count = 0;
3 }
```

Lekin hits function us variable ko reference kar raha hai, isliye closure ke through count alive rehta hai.

Condition ye hai:

Ek inner function outer scope ke variable ko use kar raha ho aur function reference outer scope ke baad bhi available rahe.

8. Closure value nahi, variable capture karta hai

```
1  var studentName = "Frank";
2
3  var greeting = function hello() {
4      console.log(`Hello, ${studentName}!`);
5  };
6
7  studentName = "Suzy";
8
9  greeting();
10 // Hello, Suzy!
```

Kuch developers expect kar sakte hain:

```
1 Hello, Frank!
```

Kyunki greeting function tab create hua tha jab:

```
1 studentName = "Frank";
```

Lekin result hota hai:

```
1 Hello, Suzy!
```

Reason:

```
1 greeting
```

ne "Frank" value capture nahi ki.

Usne studentName variable se live connection preserve kiya.

Later variable update hua:

```
1 studentName = "Suzy";
```

Jab function invoke hua, usne studentName ki latest value read ki.

```
1 Closure → studentName variable
2 studentName variable → current value "Suzy"
```

9. Classic var loop closure problem

```
1 var keeps = [];
```

```

2
3  for (var i = 0; i < 3; i++) {
4      keeps[i] = function keepI() {
5          return i;
6      };
7  }
8
9  keeps[0](); // 3
10 keeps[1](); // 3
11 keeps[2](); // 3

```

Aap expect kar sakte ho:

```

1  0
2  1
3  2

```

Lekin milta hai:

```

1  3
2  3
3  3

```

Reason

var i ke saath poore loop ke liye sirf **ek single i variable** create hota hai.

Loop ke during:

```

1  Iteration 1: i = 0
2  Iteration 2: i = 1
3  Iteration 3: i = 2
4  Loop end:    i = 3

```

Teen functions create hue, lekin teeno same i variable ko close over kar rahe hain:

```

1  keeps[0] ┌
2  keeps[1] ┤ ──> same i variable
3  keeps[2] └

```

Functions ko loop ke complete hone ke baad call kiya gaya. Us waqt:

```

1  i === 3

```

Isliye teeno 3 return karte hain.

Important:

Teen closures hain, lekin teeno ka connection ek hi shared variable se hai.

10. Separate variable se loop problem solve karna

```
1  var keeps = [];  
2  
3  for (var i = 0; i < 3; i++) {  
4      let j = i;  
5  
6      keeps[i] = function keepEachJ() {  
7          return j;  
8      };  
9  }  
10  
11 keeps[0](); // 0  
12 keeps[1](); // 1  
13 keeps[2](); // 2
```

Har iteration me:

```
1  let j = i;
```

ek naya j variable create karta hai.

Conceptually:

```
1  Iteration 1:  
2  j1 = 0  
3  function 1 closes over j1  
4  
5  Iteration 2:  
6  j2 = 1  
7  function 2 closes over j2  
8  
9  Iteration 3:  
10 j3 = 2  
11 function 3 closes over j3
```

Isliye:

```
1  keeps[0]() // j1 → 0  
2  keeps[1]() // j2 → 1  
3  keeps[2]() // j3 → 2
```

11. for (let...) automatically problem solve karta hai

```
1  var keeps = [];  
2  
3  for (let i = 0; i < 3; i++) {  
4      keeps[i] = function keepEachI() {  
5          return i;  
6      };  
7  }  
8  
9  keeps[0](); // 0  
10 keeps[1](); // 1  
11 keeps[2](); // 2
```

for loop initialization me let use karne par JavaScript har iteration ke liye effectively ek naya i binding create karta hai.

```
1  Iteration 1: separate i = 0  
2  Iteration 2: separate i = 1  
3  Iteration 3: separate i = 2
```

Har function apne iteration wale i ko close over karta hai.

Ye modern JavaScript me preferred solution hai:

```
1  for (let i = 0; i < 3; i++) {  
2      // callback uses its own iteration's i  
3  }
```

12. Callbacks me closure

Closure sabse commonly callbacks ke saath dikhta hai.

Conceptual Ajax example:

```
1  function lookupStudentRecord(studentID) {  
2      ajax(  
3          `https://some.api/student/${studentID}`,  
4          function onRecord(record) {  
5              console.log(  
6                  `${record.name} (${studentID})`  
7              );  
8          }  
9      );
```

```
10 }  
11  
12 lookupStudentRecord(114);
```

Timeline

```
1 1. lookupStudentRecord(114) call hua  
2 2. studentID = 114  
3 3. Ajax request start hui  
4 4. onRecord callback register hua  
5 5. lookupStudentRecord() complete ho gaya  
6 6. Kuch time baad response aaya  
7 7. Ajax system ne onRecord(record) invoke kiya  
8 8. Callback ne studentID access kiya
```

Step 5 par outer function complete ho chuka tha.

Fir bhi step 8 par:

```
1 studentID
```

available hai.

Reason:

```
1 onRecord
```

callback ne studentID variable ko close over kiya.

Closures create kar sakte hain chahe callback:

- Ajax response callback ho
- timer callback ho
- promise callback ho
- event handler ho
- array callback ho

13. Event handlers me closure

```
1 function listenForClicks(btn, label) {  
2   btn.addEventListener(  
3     "click",  
4     function onClick() {  
5       console.log(  
6         `The ${label} button was clicked!`  
7       );  
8     }  
9   );  
10 }
```

```

8      }
9      );
10   }
11
12   var submitBtn = document.getElementById("submit-btn");
13
14   listenForClicks(submitBtn, "Checkout");

```

listenForClicks() complete ho jayega, lekin onClick() function browser ke event system me registered rahega.

Later jab user button click karega:

```

1  onClick()

```

run hoga.

Us waqt bhi:

```

1  label === "Checkout"

```

available rahega.

Reason:

```

1  onClick

```

ne label ko close over kiya hai.

14. Lexical scope aur observable closure me difference

Ye chapter closure ko observational perspective se define karta hai.

Example:

```

1  function say(myName) {
2      var greeting = "Hello";
3
4      output();
5
6      function output() {
7          console.log(
8              `${greeting}, ${myName}!`
9          );
10     }
11 }
12

```

```
13 say("Kyle");
```

output() outer variables access karta hai:

```
1 greeting
2 myName
```

Lekin output() ko wahi outer scope active hote waqt invoke kiya ja raha hai.

```
1 say() running
2   ↓
3 output() invoked immediately
```

Outer scope abhi finish nahi hua. Isliye observable perspective se ye normal lexical-scope lookup hai, closure ka observable effect nahi.

Simple distinction

Lexical scope

```
1 function outer() {
2     let value = 10;
3
4     function inner() {
5         console.log(value);
6     }
7
8     inner();
9 }
```

inner() outer scope active hote waqt execute hua.

Observable closure

```
1 function outer() {
2     let value = 10;
3
4     return function inner() {
5         console.log(value);
6     };
7 }
8
9 const innerFn = outer();
10
11 innerFn();
```

outer() complete hone ke baad inner() invoked hua. Fir bhi value accessible hai.

Ye closure ka observable behavior hai.

15. Global variables ko usually closure kyon nahi kehte?

```
1  var students = [  
2    { id: 14, name: "Kyle" }  
3  ];  
4  
5  function getFirstStudent() {  
6    return function firstStudent() {  
7      return students[0].name;  
8    };  
9  }  
10  
11  var student = getFirstStudent();  
12  
13  student(); // Kyle
```

firstStudent() apne outer variable students ko access kar raha hai. Lekin students global scope me hai.

Global variables program ke har nested scope se normally accessible hote hain.

Isliye function ko students access karne ke liye special preservation ki zarurat nahi:

```
1  Global scope  
2  └─ students  
3  └─ getFirstStudent  
4      └─ firstStudent
```

firstStudent chahe kahin bhi call ho, global scope uski scope chain ka ancestor hoga.

Is chapter ke observational definition ke according ye meaningful closure example nahi, balki normal lexical lookup hai.

Technical discussions me kabhi-kabhi developers global variable access ko bhi closure bol dete hain, lekin author observable distinction maintain kar raha hai.

16. Variable present hona enough nahi hai

```
1  function lookupStudent(studentID) {  
2    return function nobody() {  
3      var msg = "Nobody's here yet.";
```

```
4     console.log(msg);
5   };
6 }
7
8 var student = lookupStudent(112);
9
10 student();
```

Outer scope me studentID present hai:

```
1 studentID = 112
```

Lekin inner function use hi nahi karta:

```
1 function nobody() {
2   var msg = "...";
3   console.log(msg);
4 }
```

Isliye nobody() ko studentID preserve karne ki need nahi.

Closure ke liye outer variable ka function ke andar reference hona zaroori hai.

Sirf variable outer scope me available hone se closure nahi banta.

17. Inner function invoke na ho to closure observe nahi hota

```
1 function greetStudent(studentName) {
2   return function greeting() {
3     console.log(
4       `Hello, ${studentName}!`
5     );
6   };
7 }
8
9 greetStudent("Kyle");
```

Outer function called hai.

Inner function create aur return bhi hua, lekin returned function ko save ya invoke nahi kiya:

```
1 greetStudent("Kyle");
```

Result directly discard ho gaya.

Isliye program me closure ka meaningful effect observe nahi hua.

Agar aise likhte:

```
1 var greetKyle = greetStudent("Kyle");  
2 greetKyle();
```

tab clearly closure observe hota.

18. Closure ki chapter wali observable definition

Chapter ke according:

Closure tab observe hota hai jab ek function outer scope ke variables ko aisi jagah execute hote hue use kare jahan wo variables otherwise directly accessible nahi hote.

Teen main conditions:

1. Function involved hona chahiye

Object ya scope alone closure nahi hai.

2. Outer variable reference hona chahiye

```
1 function inner() {  
2     return outerVariable;  
3 }
```

3. Function different scope-chain branch se invoke ho

Typical pattern:

```
1 function outer() {  
2     let value = 10;  
3  
4     return function inner() {  
5         return value;  
6     };  
7 }  
8  
9 const fn = outer();  
10 fn();
```

19. Closure lifecycle aur Garbage Collection

Closure kitni der tak zinda rehta hai?

Jab tak closed-over function ka reference program me available hai, uska required closure environment preserve reh sakta hai.

Example:

```
1 function createValue() {  
2     let value = 100;  
3  
4     return function getValue() {  
5         return value;  
6     };  
7 }  
8  
9 let fn = createValue();
```

Ab:

```
1 fn
```

returned function ko reference kar raha hai.

Isliye value ko cleanup nahi kiya ja sakta.

Later:

```
1 fn = null;
```

Agar function ka koi aur reference nahi hai, function aur value future Garbage Collection ke liye eligible ho sakte hain.

Multiple functions same variable close over karein

Agar 10 functions ek same variable ko close over karte hain:

```
1 Function 1 ┌  
2 Function 2 ┤  
3 ...       ┴── shared variable  
4 Function 10 ┘
```

9 functions ke references remove karne par bhi variable alive rahega, kyunki ek function abhi bhi us variable ko reference kar raha hai.

Jab final function reference bhi remove ho jayega, variable GC ke liye eligible ho sakta hai.

20. Event listeners memory hold kar sakte hain

Chapter ka example conceptually:

```

1  function manageBtnClickEvents(btn) {
2      var clickHandlers = [];
3
4      return function listener(cb) {
5          if (cb) {
6              let clickHandler = function onClick(evt) {
7                  console.log("clicked!");
8                  cb(evt);
9              };
10
11             clickHandlers.push(clickHandler);
12             btn.addEventListener("click", clickHandler);
13         }
14         else {
15             for (let handler of clickHandlers) {
16                 btn.removeEventListener("click", handler);
17             }
18
19             clickHandlers = [];
20         }
21     };
22 }

```

Setup:

```

1  var onSubmit = manageBtnClickEvents(mySubmitBtn);
2
3  onSubmit(function checkout(evt) {
4      // handle checkout
5  });
6
7  onSubmit(function trackAction(evt) {
8      // analytics
9  });

```

Har onClick() event handler apne cb parameter ko close over karta hai.

First handler ke liye:

```
1  cb → checkout function
```

Second handler ke liye:

```
1  cb → trackAction function
```

Jab tak event listeners registered hain, browser ke paas handler references hain. Isliye:

- onClick functions alive hain
- unke closures alive hain
- checkout aur trackAction ke references alive hain

Cleanup:

```
1 onSubmit();
```

No callback pass karne par:

1. Event listeners remove hote hain
2. clickHandlers array empty hota hai
3. Handler references discard hote hain
4. Unke closures GC ke liye eligible ho sakte hain

Practical lesson

Event listener add karna hi important nahi hai. Jab listener ki need khatam ho jaye, use remove karna bhi important hai:

```
1 element.removeEventListener("click", handler);
```

Especially:

- Single Page Applications
 - modals
 - dynamically removed components
 - long-running dashboards
 - repeated subscriptions
 - WebSocket listeners
 - custom event systems
-

21. Closure per-variable hai ya per-scope?

Conceptually hum kehte hain:

Function sirf un variables ko close over karta hai jinhe wo actually reference karta hai.

Example:

```
1 function outer() {
2     let used = 10;
3     let unused = 20;
4
5     return function inner() {
6         return used;
```

```
7     };  
8 }
```

Conceptually inner() sirf:

```
1 used
```

ko close over karta hai, unused ko nahi.

Lekin implementation level par JavaScript engine poora scope environment preserve kar sakta hai, aur baad me optimization ke through unused variables remove kar sakta hai.

Isliye do perspectives hain:

Conceptual perspective

Closure per-variable hai.

Implementation perspective

Engine poora scope preserve kar sakta hai, phir unused variables optimize out kar sakta hai.

22. manageStudentGrades() example

```
1  function manageStudentGrades(studentRecords) {  
2      var grades = studentRecords.map(getGrade);  
3  
4      return addGrade;  
5  
6      function getGrade(record) {  
7          return record.grade;  
8      }  
9  
10     function sortAndTrimGradesList() {  
11         grades.sort(function desc(g1, g2) {  
12             return g2 - g1;  
13         });  
14  
15         grades = grades.slice(0, 10);  
16     }  
17  
18     function addGrade(newGrade) {  
19         grades.push(newGrade);  
20         sortAndTrimGradesList();  
21         return grades;  
}
```

```
22     }  
23 }
```

Usage:

```
1  var addNextGrade = manageStudentGrades([  
2    { id: 14, name: "Kyle", grade: 86 },  
3    { id: 73, name: "Suzy", grade: 87 },  
4    { id: 112, name: "Frank", grade: 75 },  
5    { id: 6, name: "Sarah", grade: 91 }  
6  ]);  
7  
8  addNextGrade(81);  
9  addNextGrade(68);
```

Kaunse closures hain?

addGrade() closes over grades

```
1  function addGrade(newGrade) {  
2    grades.push(newGrade);  
3  }
```

Isliye grades outer function complete hone ke baad bhi preserve rehta hai.

addGrade() closes over sortAndTrimGradesList

```
1  sortAndTrimGradesList();
```

Ye identifier outer scope me hai aur addGrade() usko access karta hai.

Isliye function reference bhi available rehna chahiye.

sortAndTrimGradesList() closes over grades

```
1  grades.sort(...);  
2  grades = grades.slice(0, 10);
```

Important point:

```
1  grades = grades.slice(0, 10);
```

grades variable ko reassign kar raha hai. Closure array value ki snapshot par nahi, grades variable par hai. Isliye addGrade() ko next time updated array milta hai.

Kya studentRecords closed over hai?

Inner functions directly studentRecords ko reference nahi kar rahe.

Conceptually:

```
1  studentRecords
```

outer function complete hone ke baad GC ke liye available hona chahiye.

Modern engines isko optimize kar sakte hain. Lekin specification har situation me exact memory optimization guarantee nahi karti.

23. eval() closure optimization ko difficult bana deta hai

```
1 function storeStudentInfo(id, name, grade) {
2     return function getInfo(whichValue) {
3         var val = eval(whichValue);
4         return val;
5     };
6 }
7
8 var info = storeStudentInfo(73, "Suzy", 87);
9
10 info("name"); // Suzy
11 info("grade"); // 87
```

getInfo() explicitly ye nahi likh raha:

```
1 return name;
```

Ya:

```
1 return grade;
```

Instead:

```
1 eval(whichValue)
```

Runtime par dynamically decide hota hai kaunsa identifier access karna hai.

```
1 info("name");
```

eval("name") outer name variable access kar सकता है.

```
1 info("grade");
```

eval("grade") outer grade access kar सकता है.

JavaScript engine compile time par confidently determine nahi kar sakta ki:

- id needed hai ya nahi
- name needed hai ya nahi
- grade needed hai ya nahi

Isliye engine ko poora scope preserve karna pad sakta hai.

Yahi ek reason hai ki eval() problematic mana jata hai:

- code readability kharab karta hai
 - security risks create kar sakta hai
 - scope analysis difficult karta hai
 - JS engine optimizations ko prevent kar sakta hai
-

24. Large values ki memory manually release karna

Suppose studentRecords bahut bada array hai:

```
1 function manageStudentGrades(studentRecords) {  
2     var grades = studentRecords.map(getGrade);  
3  
4     studentRecords = null;  
5  
6     return addGrade;  
7  
8     // ...  
9 }
```

Yahan:

```
1 studentRecords = null;
```

variable ko scope se remove nahi karta.

Lekin variable ka large array ke saath reference break kar deta hai:

```
1 Before:  
2 studentRecords → large array  
3  
4 After:  
5 studentRecords → null
```

Agar large array ko koi aur reference nahi kar raha, to wo GC ke liye eligible ho sakta hai.

Kya har function me aisa karna chahiye?

Nahi.

Normal small variables ke liye manually null karna unnecessary hai. Ye tab consider kiya ja sakta hai jab:

- data bahut large ho
- closure very long-lived ho
- memory profiling actual problem dikhaye
- low-memory devices target ho

- function repeatedly large objects process karta ho

Pehle profiling honi chahiye, uske baad optimization.

25. Closure ka traditional mental model

Traditional model ke according:

```

1  function adder(num1) {
2      return function addTo(num2) {
3          return num1 + num2;
4      };
5  }
6
7  var add10To = adder(10);

```

Hum imagine karte hain:

1. addTo() function outer scope me return hua
2. Function apne original scope ke liye hidden link lekar gaya
3. Hidden link ke through num1 access karta hai

```

1  add10To function
2      |
3      └── hidden closure link ───> num1 = 10

```

Ye model learning ke liye intuitive hai:

Function jahan bhi jaye, apne lexical environment ko yaad rakhta hai.

26. Alternative implementation-oriented model

Alternative model kehta hai ki function actually kahin move nahi karta.

Function instance apne original scope environment me hi conceptually rehta hai:

```

1  adder execution scope
2  └─ num1 = 10
3  └─ addTo function instance

```

Global scope ko sirf function ka reference milta hai:

```

1  Global add10To
2      |

```

```
3      └─ original addTo function instance
4      |
5      └─ original num1
```

Jab:

```
1  add10To(15);
```

call hota hai, reference ke through original function invoke hota hai.

Function apne original lexical environment me hi associated hai, isliye naturally num1 access kar leta hai.

Is perspective me closure ka magic ye nahi ki function apna scope lekar travel karta hai.

Closure ka magic ye hai:

Jab tak function instance ka external reference alive hai, function aur uska required scope environment alive rakha jata hai.

Dono models same behavior explain karte hain:

Observational model

Function outer variables ko remember karta hai.

Implementation model

Function instance aur uska scope environment alive rehte hain, external code ko sirf function reference milta hai.

Beginner ke liye observational model usually easier hai.

27. Closure use kyon karein?

Closure ke do major benefits hain:

Benefit 1: Encapsulation

Variables ko global banane ke bajay private outer scope me rakh sakte ho.

```
1  function makeCounter() {
2      let count = 0;
3
4      return function increment() {
5          count++;
6          return count;
7      };
}
```

```
8 }
```

Outside code directly count access nahi kar sakta:

```
1 const counter = makeCounter();  
2  
3 counter(); // 1  
4 counter(); // 2
```

count protected aur narrowly scoped hai.

Benefit 2: Information remember karna

Function future calls ke liye previously calculated information remember kar sakta hai.

```
1 function multiplyBy(multiplier) {  
2     return function multiply(number) {  
3         return number * multiplier;  
4     };  
5 }  
6  
7 const double = multiplyBy(2);  
8 const triple = multiplyBy(3);  
9  
10 double(5); // 10  
11 triple(5); // 15
```

double ko har baar 2 pass nahi karna padta.

triple ko har baar 3 pass nahi karna padta.

Benefit 3: Repeated work avoid karna

Configuration ko ek baar calculate karke closure me preserve kar sakte hain.

Benefit 4: Specialized functions banana

```
1 const addTax = createTaxCalculator(0.18);  
2 const addDiscount = createDiscountCalculator(0.10);
```

Functions meaningful kaam represent karte hain aur required configuration remember rakhte hain.

28. Button handler without effective closure

```
1 function makeRequest(evt) {  
2     var btn = evt.target;
```

```

3     var recordKind = btn.dataset.kind;
4
5     ajax(
6         APIEndpoints[recordKind],
7         data[recordKind]
8     );
9 }
10
11 btn.addEventListener("click", makeRequest);

```

Har click par function:

1. Event se button nikalega
2. DOM se data-kind attribute read karega
3. Endpoint lookup karega
4. Data lookup karega

Ye kaam karta hai, lekin repeated DOM lookup required hai.

29. Closure-based button handler

```

1  function setupButtonHandler(btn) {
2      var recordKind = btn.dataset.kind;
3
4      btn.addEventListener(
5          "click",
6          function makeRequest(evt) {
7              ajax(
8                  APIEndpoints[recordKind],
9                  data[recordKind]
10             );
11         }
12     );
13 }
14
15 setupButtonHandler(btn);

```

Ab setup ke time hi:

```

1  var recordKind = btn.dataset.kind;

```

read hota hai.

makeRequest() closure ke through recordKind remember karta hai.

Jab click hoga, function ko DOM attribute dobara read nahi karna padega.

Benefits:

- variable global nahi banana pada
 - repeated DOM read avoid hua
 - event handler simpler hua
 - recordKind sirf relevant code tak limited raha
-

30. URL aur data bhi pehle calculate karna

```
1 function setupButtonHandler(btn) {
2     var recordKind = btn.dataset.kind;
3     var requestURL = APIEndpoints[recordKind];
4     var requestData = data[recordKind];
5
6     btn.addEventListener(
7         "click",
8         function makeRequest(evt) {
9             ajax(requestURL, requestData);
10        }
11    );
12 }
```

Setup ke time:

```
1 requestURL
2 requestData
```

calculate hote hain.

Event handler in dono ko close over karta hai.

Later click par direct:

```
1 ajax(requestURL, requestData);
```

call hota hai.

Ye tab useful hai jab setup ke baad corresponding endpoint/data mapping change nahi honi chahiye.

Lekin closure live variables preserve karta hai. Agar requestData kisi object ko point karta hai aur object mutate hota hai, handler ko mutated object mil sakta hai.

Example:

```

1  const requestData = {
2      id: 1
3  };
4
5  function handler() {
6      console.log(requestData.id);
7  }
8
9  requestData.id = 2;
10
11 handler(); // 2

```

Variable agar same object ko refer kar raha hai, object ke internal changes visible rahenge.

31. Partial application using closure

```

1  function defineHandler(requestURL, requestData) {
2      return function makeRequest(evt) {
3          ajax(requestURL, requestData);
4      };
5  }

```

Then:

```

1  function setupButtonHandler(btn) {
2      var recordKind = btn.dataset.kind;
3
4      var handler = defineHandler(
5          APIEndpoints[recordKind],
6          data[recordKind]
7      );
8
9      btn.addEventListener("click", handler);
10 }

```

defineHandler() ko pehle hi kuch inputs diye gaye:

```

1  requestURL
2  requestData

```

Ye inputs returned function closure ke through remember karta hai.

Later event ke time final input aata hai:

```

1  evt

```

Although example me evt use nahi ho raha.

Is pattern ko broadly partial application kaha ja sakta hai:

Multi-input operation ke kuch inputs pehle provide karo aur baaki inputs later provide karo.

Simple example:

```
1 function multiply(firstNumber) {
2   return function by(secondNumber) {
3     return firstNumber * secondNumber;
4   };
5 }
6
7 const double = multiply(2);
8
9 double(5); // 10
10 double(10); // 20
```

2 pehle provide hua aur closure me preserve hua. Doosra input baad me diya gaya.

32. Partial application aur currying

Chapter briefly dono ka mention karta hai.

Partial application

Kisi function ke kuch arguments pehle fix karna.

Conceptual example:

```
1 function calculatePrice(price, tax, discount) {
2   return price + price * tax - discount;
3 }
```

Partial application style:

```
1 function createCalculator(tax, discount) {
2   return function calculate(price) {
3     return price + price * tax - discount;
4   };
5 }
6
7 const indiaCalculator = createCalculator(0.18, 100);
```

```
8
9  indiaCalculator(1000);
```

tax aur discount closure me preserved hain.

Currying

Multi-parameter function ko chained single-parameter functions me convert karna:

```
1  function add(a) {
2      return function addB(b) {
3          return function addC(c) {
4              return a + b + c;
5          };
6      };
7  }
8
9  add(1)(2)(3); // 6
```

Yahan:

- addB closes over a
- addC closes over a and b

Closure currying aur partial application dono ka foundational mechanism hai.

33. Closure aur private state

Closure modules aur private state banane me important hai.

```
1  function createBankAccount(initialBalance) {
2      let balance = initialBalance;
3
4      return {
5          deposit(amount) {
6              balance += amount;
7              return balance;
8          },
9
10         getBalance() {
11             return balance;
12         }
13     };
14 }
```

```
15
16 const account = createBankAccount(1000);
17
18 account.deposit(500); // 1500
19 account.getBalance(); // 1500
```

balance directly outside accessible nahi:

```
1 account.balance; // undefined
```

Lekin methods balance ko close over karte hain:

```
1 deposit ———┐
2               └───> balance
3 getBalance ———┐
```

Object khud closure nahi rakhta. Object ke andar jo functions hain, wo closure rakhte hain.

Ye pattern module pattern ka base hai, jise next chapter explore karega.

34. Closure ke common mistakes

Mistake 1: Closure value capture karta hai

Wrong thinking:

```
1 Function create hua jab value 10 thi
2 Isliye function hamesha 10 dekhega
```

Correct:

```
1 Function variable ko reference karta hai
2 Later variable ki current value dekhega
```

Mistake 2: Har loop iteration ka var i separate hai

Wrong:

```
1 for (var i = 0; i < 3; i++) {
2     // separate i?
3 }
```

Actually single shared i hai.

Correct solution:

```
1 for (let i = 0; i < 3; i++) {
2     // new binding per iteration
```

```
3 }
```

Mistake 3: Outer scope ke saare variables necessarily forever retained hain

Conceptually function required referenced variables close over karta hai. Engines unused variables optimize kar sakte hain.

Lekin large values aur long-lived closures ke case me memory profile karna useful hai.

Mistake 4: Closure ka matlab asynchronous code hai

Closure async hona required nahi.

```
1 function outer(x) {  
2     return function inner(y) {  
3         return x + y;  
4     };  
5 }
```

Ye closure hai, bina Ajax, timer ya event ke.

Asynchronous callbacks me closure easily observable hota hai, isliye wahan common hai.

Mistake 5: Closure create hote hi memory leak hota hai

Closure itself memory leak nahi hai.

Problem tab hoti hai jab:

- unnecessary function references retain kiye jayein
- event listeners remove na kiye jayein
- timers cleanup na kiye jayein
- large objects unnecessary closures me retained rahein
- dead UI components se callbacks attached rahein

Closure intentional state preservation hai. Memory leak unintentional retention hai.

35. Closure identify karne ka practical method

Kisi function ko dekhkar ye questions pucho:

Question 1: Function apne local parameters/variables ke alawa kuch use kar raha hai?

```
1 function inner() {  
2     return outerValue;  
3 }
```

Haan, outerValue.

Question 2: Wo variable kahan declare hua hai?

Agar lexical outer scope me declare hua hai, closure candidate hai.

Question 3: Inner function kab aur kahan execute hoga?

Agar outer function complete hone ke baad execute hota hai:

- returned function
- event handler
- timer callback
- Ajax callback
- Promise callback
- stored callback

to closure clearly observable hai.

Question 4: Function reference kitni der alive rahega?

Jab tak reference exists hai, captured environment retained reh sakta hai.

Question 5: Kya closure large data retain kar raha hai?

Agar haan, evaluate karo:

- kya data actually required hai?
- listener cleanup ho raha hai?
- function reference discard ho raha hai?
- data ko smaller form me store kar sakte hain?
- memory profiling issue confirm karti hai?

36. Ek complete practical example

```
1 function createUserGreeter(userName) {  
2     let greetingCount = 0;  
3 }
```

```

4     return function greet() {
5         greetingCount++;
6
7         return `Hello ${userName}, greeted ${greetingCount}
times`;
8     };
9 }
10
11 const greetRuth = createUserGreeter("Ruth");
12 const greetKyle = createUserGreeter("Kyle");
13
14 console.log(greetRuth());
15 // Hello Ruth, greeted 1 times
16
17 console.log(greetRuth());
18 // Hello Ruth, greeted 2 times
19
20 console.log(greetKyle());
21 // Hello Kyle, greeted 1 times

```

greetRuth closure

```

1  userName = "Ruth"
2  greetingCount = 2

```

greetKyle closure

```

1  userName = "Kyle"
2  greetingCount = 1

```

Dono same greet() definition se aaye hain, lekin separate function instances aur separate scope environments hain.

Is example me closure:

- userName configuration remember karta hai
- greetingCount mutable state remember karta hai
- variables ko global hone se bachata hai
- har greeter ko independent state deta hai

37. Chapter ka final mental model

Closure ko simplest words me aise samjho:

Jab ek function apne outer scope ke variable ko use karta hai, aur wo function future me outer scope ke complete hone ke baad bhi available rehta hai, JavaScript us function ko required variable tak access preserve karke deta hai. Isi preserved access ko closure kehte hain.

Visual model:

```
1 function outer() {  
2     let data = 10;  
3  
4     return function inner() {  
5         return data;  
6     };  
7 }  
8  
9 const fn = outer();
```

Conceptually:

```
1 fn  
2 |  
3 └─→ inner function  
4     |  
5     └─→ data variable  
6         |  
7         └─→ 10
```

Agar data update kiya ja sakta ho:

```
1 fn  
2 |  
3 └─→ inner function  
4     |  
5     └─→ same live data variable
```

Ye data ki photo nahi hai. Ye actual variable ka live connection hai.

38. Dono closure models ka final comparison

Observational model

Function outer variables ko remember karta hai, even when function ko kisi aur scope se invoke kiya jata hai.

Learning aur code behavior understand karne ke liye ye model best hai.

Implementation-oriented model

Function instance apne scope environment ke saath alive rehta hai, jab tak us function ka koi reference available hai.

Memory management aur references samajhne ke liye ye model helpful hai.

Dono ke observable results same hain.

39. Complete chapter summary

1. **Closure sirf functions ka behavior hai.**
 2. **Function outer lexical scope ke variables ko access kar sakta hai.**
 3. **Closure tab clearly observe hota hai jab function outer scope complete hone ke baad invoke hota hai.**
 4. **Closure value ki snapshot nahi rakhta. Ye variable ke saath live connection rakhta hai.**
 5. **Variable ko closure ke through read aur update dono kiya ja sakta hai.**
 6. **Har function instance ka separate closure ho sakta hai.**
 7. **var loop me callbacks same variable share karte hain.**
 8. **let loop me har iteration ke liye separate binding banti hai.**
 9. **Callbacks, event handlers, timers aur Ajax me closure commonly use hota hai.**
 10. **Jab tak function reference alive hai, required captured variables bhi alive reh sakte hain.**
 11. **Unused listeners aur callbacks memory ko unnecessarily retain kar sakte hain.**
 12. **Closure encapsulation, private state, partial application, currying aur module pattern ka foundation hai.**
 13. **Closure code ko efficient bana sakta hai, kyunki previously determined information repeat calculate ya pass nahi karni padti.**
 14. **Closure scope exposure kam karta hai, kyunki variables ko global banaye bina future me use kiya ja sakta hai.**
-

One-line definition

Closure tab hota hai jab koi function apne lexical outer scope ke variables ka live access preserve rakhta hai, even after us outer scope ka normal execution complete ho chuka ho.

Apne words me restatement

Closure function ki ek tarah ki memory hai, lekin wo kisi value ki fixed copy nahi hai. Function apne creation wale lexical environment ke required variables ke saath connected rehta hai. Jab function ko baad me kisi doosri jagah se call kiya jata hai, tab bhi wo un variables ko read aur modify kar sakta hai. Isi wajah se hum private state, reusable configured functions, event handlers, counters aur modules jaise patterns bana sakte hain.